

Introduction to Machine Learning (67577)

Exercise 3 Gradient-Based Learning and Neural Networks

Second Semester, 2026

Contents

1	Theoretical Part	2
1.1	Convex optimization	2
1.2	Sub-gradients for Soft-SVM Objective	2
1.3	Feed Forward Neural Networks	3
2	Practical Part	4
2.1	Gradient Descent	4
2.1.1	Comparing Fixed learning rates	4
2.1.2	Comparing Exponentially Decaying learning rates	5
2.2	Feed-Forward Neural Networks	5
2.2.1	Classifying a 2D linearly inseparable simulated dataset	7
2.2.2	Classifying MNIST with Feed-Forward Neural Network You Have Implemented from Scratch	8

Submission

Please make sure to follow the general submission instructions available on the course website. In addition, for the following assignment, submit a single `ex3_ID.zip` file containing:

- An `Answers.pdf` file with the answers for all theoretical and practical questions (include plotted graphs *in* the PDF file).
- The following python files (without any directories):
`gradient_descent.py`, `gradient_descent_investigation.py`,
`learning_rate.py` `modules.py`, `neural_network.py`,
`nn_loss_functions.py`, `nn_mnist_digit_classification.py`,
`nn_simulated_data.py`, `nn_utils.py`,
`nn_modules.py`,
`stochastic_gradient_descent.py` ,

The `ex3_ID.zip` file must be submitted in the designated Moodle activity prior to the date specified *in the activity*.

- Plots included as separate files will be considered as not provided.

1 Theoretical Part

1.1 Convex optimization

1. Let $f_1, \dots, f_m : C \rightarrow \mathbb{R}$ be a set of convex functions and $\gamma_1, \dots, \gamma_m \in \mathbb{R}_+$. Prove from definition that $g(\mathbf{u}) = \sum_{i=1}^m \gamma_i f_i(\mathbf{u})$ is a convex function.
2. Give a counterexample for the following claim: Given two functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$, define a new function $h : \mathbb{R} \rightarrow \mathbb{R}$ by $h = f \circ g$. If f and g are convex then h is convex as well.

1.2 Sub-gradients for Soft-SVM Objective

The Soft-SVM objective, though convex, is not differentiable in all of its domain due to the use of the hinge-loss. Therefore, to implement a sub-gradient descent solver for this problem we must first describe sub-gradients of the objective.

3. Given $\mathbf{x} \in \mathbb{R}^d$ and $y \in \{\pm 1\}$. Show that the hinge loss is convex in $\mathbf{w}, b$. That is, define

$$f(\mathbf{w}, b) := \ell_{\mathbf{x}, y}^{\text{hinge}}(\mathbf{w}, b) = \max(0, 1 - y(\mathbf{x}^\top \mathbf{w} + b))$$

and show that f is convex in $\mathbf{w}, b$.

4. Deduce some sub-gradient of the hinge loss function $g \in \partial \ell_{\mathbf{x}, y}^{\text{hinge}}(\mathbf{w}, b)$.
5. Let $f_1, \dots, f_m : \mathbb{R}^d \rightarrow \mathbb{R}$ be a set of convex functions and $\mathbf{g}_k \in \partial f_k(\mathbf{x})$ for all $k \in [m]$ be sub-gradients of these functions. Define $f : \mathbb{R}^d \rightarrow \mathbb{R}$ by $f(\mathbf{x}) = \sum_{i=1}^m f_i(\mathbf{x})$. Show that $\sum_k \mathbf{g}_k \in \partial \sum_k f_k(\mathbf{x})$.
6. Let $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m \subseteq \mathbb{R}^d \times \{\pm 1\}$ be a sample and define $f : \mathbb{R}^d \rightarrow \mathbb{R}$ by:

$$f(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m \ell_{\mathbf{x}_i, y_i}^{\text{hinge}}(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Find a sub-gradient of f for any $\mathbf{w}$.

1.3 Feed Forward Neural Networks

7. Consider a simple feedforward neural network with the following architecture:
 - Input: $x \in \mathbb{R}^d$
 - Hidden layer: $h = \sigma(Wx + b)$, where $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$, and σ is the elementwise ReLU activation
 - Output: $\hat{y} = u^T h + c$, where $u \in \mathbb{R}^m$ and $c \in \mathbb{R}$ (i.e., scalar output)
 - Loss: $L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$
 - (a) Derive the gradient of the loss with respect to the output layer parameters u and c .
 - (b) Derive the gradient with respect to the hidden layer weights W and biases b .
 - (c) Which parts of the computation require storing intermediate values during the forward pass in order to compute the gradients correctly during backpropagation?
8. The most common loss function used in classification tasks is the **Cross-Entropy (CE) loss**. To understand its application, we first define the Cross-Entropy between two discrete probability distributions, $\mathbf{p}$ (the true distribution) and $\mathbf{q}$ (the predicted distribution), as:

$$CE(\mathbf{p}, \mathbf{q}) = - \sum_{j=1}^k p_j \log(q_j)$$

In the context of a neural network, the raw outputs (the **logits**) are denoted by $\mathbf{x}$. These are passed through the **Softmax** function to produce a predicted probability distribution $\mathbf{z} = S(\mathbf{x})$. For a single training sample belonging to class i , the ground-truth is represented by a **one-hot vector** $\mathbf{e}_i$ (where the i -th element is 1 and all others are 0).

In this specific case, the Cross-Entropy loss simplifies significantly:

$$L_i(\mathbf{z}) = CE(\mathbf{e}_i, \mathbf{z}) = - \sum_{j=1}^k (\mathbf{e}_i)_j \log(z_j) = -\log(z_i)$$

In this question, we will calculate the Jacobian of the composition of the Cross-Entropy function with the Softmax function. You will use this derivative in the practical part of this exercise.

Recall the Softmax function $S: \mathbb{R}^k \rightarrow [0, 1]^k$, defined as:

$$S(\mathbf{x})_j = \frac{e^{x_j}}{\sum_{l=1}^k e^{x_l}}$$

Recall that we calculated its Jacobian in Ex1, which is:

$$J_{\mathbf{x}}(S) = \text{diag}(S(\mathbf{x})) - S(\mathbf{x})S(\mathbf{x})^\top$$

- a) Calculate the Jacobian of the Cross-Entropy loss with respect to the probabilities, $J_{\mathbf{z}}(L_i)$.
- b) Apply the multivariable chain rule to calculate the Jacobian of the loss with respect to the logits: $J_{\mathbf{x}}(L_i \circ S) = J_{\mathbf{z}}(L_i) \cdot J_{\mathbf{x}}(S)$.

2 Practical Part

2.1 Gradient Descent

In the following section you will implement a generic Gradient Descent algorithm, and explore and visualize its performance on different two objective functions - L1 and L2. To assist you with the implementation please start by reading the documentation of the `gradient_descent.py` file and following the steps as described below.

Learning Rate: The `GradientDescent` class, when initialized, receives a *learning rate strategy* in the form of a `BaseLR` instance. Read the documentation of the `BaseLR` base class in the `learning_rate.py` file and then implement the learning rate strategies in the :

- Constant (Fixed) Learning Rate (i.e. $\eta_t = \eta$) - `FixedLR` class in the `learning_rate.py` file
- Exponentially decaying Learning Rate (i.e. $\eta_t = \eta \cdot \gamma^t$) - `ExponentialLR` class in the `learning_rate.py` file

Objective Functions (Modules): When running the `GradientDescent.fit` function it receives an instance derived from the `BaseModule` class. This class defines the generic abstract form of any objective to be minimized using gradient descent. Its two main functions are used to compute the value of the function and the derivative of the function at a given point of interest. Read the documentation of the `BaseModule` base class in the `base_module.py` file.

Implement the L2 and L1 modules in the `modules.py` file. Note that both these modules ignore any passed inputs in the `compute_output` and `compute_jacobian` functions and simply use the weights defined in the base class.

Gradient Descent Algorithm Implement the `GradientDescent` class in the `gradient_descent.py` file.

- Note that when instantiating a `GradientDescent` object a `callback` can be passed. This will be used to investigate different properties of the algorithm's run and will be specified in the questions below.
- Implementation must support several solution types, one of which is the average of $\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(t)}$. In your implementation do not store all solutions to avoid wasting memory.

2.1.1 Comparing Fixed learning rates

We will get the feeling of how different parameters affect the learning process and what is the meaning of learning rate. We begin with investigating the GD convergence over the L1 and L2 objectives using fixed learning rates. In the `gradient_descent_investigation.py` file implement the function `compare_fixed_learning_rates` as specified in function documentation:

- Implement the `get_gd_state_recorder_callback` function as specified in function documentation. This function returns a “fresh” callback function and lists for losses and weights throughout the GD iterations.
- Minimize the L1 and L2 modules for each of the following fixed learning rates $\eta \in \{1, 0.1, 0.01, 0.001\}$, setting the initial starting point (i.e. the initial value of the module's weights) to $\mathbf{w}_0 = \left(\sqrt{2}, \frac{e}{3}\right)$

- Notice that the L2 module actually implements the **squared** L2 norm.

Of note, all the objective functions we saw so far depended on the training data $\mathbf{X}, \mathbf{y}$. In this section, we minimize a function that ignores the given training data (like all regularization terms). Modules implemented later in this exercise will use the training data.

Then, answer the following questions:

1. Plot the descent path for each of the settings described above (you can use the `plot_descent_path`). Add below the plots for $\eta = 0.01$ and explain the differences seen between the L1 and L2 modules.
2. following the previous question describe two phenomena that you have seen in the descent path of the ℓ_1 objective when using GD and a fixed learning rate.
3. For each of the modules, plot the convergence rate (i.e. the norm as a function of the GD iteration) for all specified learning rates. Explain your results
4. What is the lowest loss achieved when minimizing each of the modules? Explain the differences

2.1.2 Comparing Exponentially Decaying learning rates

Next, we will use the exponential decay (instead of the fixed learning rate) to optimize the L1 module. Starting from $\mathbf{w}_0 = \left(\sqrt{2}, \frac{\epsilon}{3}\right)$ use the exponential decay with $\eta = 0.1$ and $\gamma \in \{0.9, 0.95, 0.99, 1\}$. Then, answer the following questions.

5. Plot the convergence rate for all decay rates in a single plot. Explain your results.
6. How does the algorithm perform using the exponential decay compared to the fixed learning rate? What is the lowest ℓ_1 norm achieved using the exponential decay. Explain why there are differences.
7. Plot the descent path for the $\gamma = 0.95$. Describe how the descent path changed from when using a fixed learning rate.

2.2 Feed-Forward Neural Networks

In this section you will implement a simple feed-forward neural network from scratch. You will then use it to learn both simulated and real-world data. You should use your implementations of Gradient Descent and learning rates from the previous section. You would also implement Stochastic Gradient Descent, which should be very similar to your Gradient Descent implementation.

Implement the following classes as described below. Follow class and function documentations.

- Implement the `cross_entropy` and `softmax` functions in the `nn_loss_functions.py` file.
- Implement the following modules in the file `nn_modules.py`: `FullyConnectedLayer`, `ReLU`, and `CrossEntropyLoss`:
 - **Forward Pass:** Each layer must cache the specific intermediate elements of the forward pass (e.g., input matrices) required for the backward pass calculation within the class instance itself.
 - **Backward Pass:** Each layer and activation must implement a `backprop` method that receives the gradient from the deeper layer (the upstream gradient) and returns the gradient with respect to the layer's input (the downstream gradient).

- **Weight Gradients:** In addition to propagating the gradient to the previous layer, each layer with trainable parameters must calculate the gradient with respect to its weights and store it in an attribute named `self._grad_weights`.
- **Memory Management:** Implement a `clear_cache` function in each class to nullify (set to `None`) any stored calculations or tensors that are no longer necessary after the gradient has been propagated.
- Implement the `NeuralNetwork` class in `neural_network.py` in the following steps:
 - Implement the `compute_output` method according to the Forward pass. Propagate the input through all network layers sequentially.
 - Implement the `compute_jacobian` method according to the Backwards pass. The network should orchestrate the process by calling the `backprop` function of each layer in reverse order.
 - **Starting the Chain:** Note that the `CrossEntropyLoss` module specifically implements a `compute_jacobian` method. This method is responsible for calculating the *initial* jacobian of the loss with respect to the network's final output. This result serves as the starting point for the backward chain.
 - **Gradient Collection:** The `compute_jacobian` method of the `NeuralNetwork` must collect the `self._grad_weights` from each layer and return a flattened vector of all weight gradients for simple usage in `GradientDescent`/`StochasticGradientDescent`. Flattening can be done using the `flatten_parameters` method.
 - **Cleanup:** Ensure that `clear_cache` is called appropriately at each layer immediately after its gradient has been collected to maintain memory efficiency.
- Implement `StochasticGradientDescent` class in `stochastic_gradient_descent.py`.
- Implement the `confusion_matrix` function in `nn_utils.py`.

Implementation tips and hints:

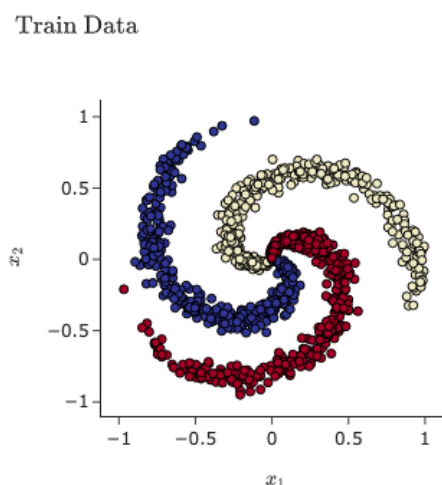
As this is a non-trivial algorithm to implement, use the following tips and hints for the backpropagation implementation. *Be sure to read these carefully.*

- **Weight Initialization:** When initializing a `FullyConnectedLayer` with d inputs and k outputs, each weight should be initialized randomly following the distribution $\mathcal{N}(0, \frac{1}{d})$.
- **Matrix Notation (Row-Major):** Note that our implementation uses the notation $\mathbf{XW}$ (where each sample is a row), whereas in the recitation we used $\mathbf{WX}$ (where each sample is a column). This affects the backpropagation implementation, particularly in how derivatives are propagated through the linear layers (see details below).
- **Efficient Backpropagation:** While the recitation derived the algorithm for single samples, your `backprop` should process the entire batch. Transform the "upstream" matrix (loss derivative w.r.t. output) into a "downstream" matrix (loss derivative w.r.t. input), while avoiding the computation of high-dimensional tensors.
 - **ReLU Layers:** Since these are element-wise, simply compute the element-wise multiplication between the upstream matrix and the local derivative.
 - **Linear Layers (Input Gradient):** In contrast to the $\mathbf{WX}$ notation from class, our $\mathbf{XW}$ setup requires multiplying the upstream gradient by the weight transpose ($\Delta \mathbf{W}^\top$) to propagate error to the previous layer.
 - **Linear Layers (Weight Gradient):** Store the weight derivative in `self._grad_weights`. Use $\mathbf{X}^\top \Delta$ to aggregate the gradient contributions from every sample in the batch into a single update.

- **Terminology:** Note that `compute_jacobian` does not return a formal Jacobian. Instead, for `CrossEntropyLoss`, it provides a matrix of partial derivatives shaped as specified in the documentation for the start of the backward pass. For `NeuralNetwork`, it flattens and concatenates all internal partial derivatives into the single vector required by the optimizer.
- **Cross-Entropy Derivative:** The implementation of the `compute_jacobian` method of `CrossEntropyLoss` should be based on the derivative you derived in the theoretical part of this exercise. Note that while you derived it for a single sample, your implementation must handle a batch of multiple samples and return the matrix of partial derivatives accordingly.
- **Additional Material:** Helpful conceptual material can be found: [D2L Backpropagation](#), [Neural Matrix Calculus](#), or [Karpathy's Backprop Explained](#).

2.2.1 Classifying a 2D linearly inseparable simulated dataset

In this part, you will use your neural network implementation to classify a simple 2d dataset as seen below, which is not linearly separable. Since this data is 2d, we can easily visualize the network's decision boundary, and see the piecewise linear behavior of the neural network.



In the `nn_simulated_data.py` file

- Create a simple architecture that consists of *two* fully connected hidden layers, each with 16 neurons, an intercept (not included in the 16 neurons) and ReLU activations.
- Specify the network's loss function to be Softmax-Cross Entropy loss function.
- Specify the network's solver to be your previously implemented gradient descent algorithm set to use a fixed learning rate of $\eta \equiv 0.1$ and to perform up to 5000 iterations

Then, answer the following questions:

1. Fit the network over the train set using the hyperparameters mentioned above. Using the `plot_decision_boundary` function, plot the boundaries learned by the network. Then, evaluate network's performance (accuracy) over the test set and report the results.
2. Remove both hidden layers (only for this question), and repeat the previous question. Explain the results.

3. Rerun the fitting of the network in question 1, this time while specifying a callback function to record network's convergence process. Plot the convergence process. i.e., loss as a function of iteration and well as gradient norm as a function of iteration. In addition, on every 100'th iteration store network's weights. Call the `animate_decision_boundary` function to see how the network's decision boundaries change as fitting progresses. No need to include the outputted animation in your submission.
4. Decrease the hidden layer's width to 6 neurons each (only for this question), and repeat the previous question (both plot and animation). Explain the results.

2.2.2 Classifying MNIST with Feed-Forward Neural Network You Have Implemented from Scratch

Next, using the `NeuralNetwork` class you have implemented from scratch, we turn to classify a more challenging task—the MNIST image recognition dataset. As this dataset is much larger, using GD is computationally challenging. Therefore, we will use Stochastic Gradient Descent (SGD) in order to improve runtime.

In the `nn_mnist_digit_classification.py` file

- Create a simple architecture that consists of *two* fully connected hidden layers, each with 64 neurons, an intercept (not included in the 64 neurons) and ReLU activations.
- Specify the network's loss function to be the Softmax-Cross Entropy loss function.
- Specify the network's solver to be your implementation of the SGD algorithm set to use a fixed learning rate of $\eta \equiv 0.1$, perform up to 10,000 iterations and to use batches of 256 samples.
- Specify a callback function that records convergence process: at each call the current loss and the gradient norm.

After completing those, answer the following questions:

5. Train the neural network on the train set using the hyperparameters mentioned above. Evaluate it on the test set and report the test accuracy.
6. Plot the convergence process. i.e., loss as a function of iteration and well as gradient norm as a function of iteration.
7. Plot a confusion matrix between the true- and predicted labels of the test set. What are the two most common confusion? What are the three least common confusions? Do these results make sense?
8. Remove both hidden layers (only for this question), and repeat question 5. What can be assumed about the data based on the evaluated test accuracy?
9. Recall that the prediction of a network (after Softmax) is a probability vector $\mathbf{p}$, where p_k describes the probability that the true class is k . We define the *confidence* of a prediction $\mathbf{p}$ as $\max_k p_k$. Notice that if $\mathbf{p}$ is the uniform vector, the confidence is minimal.
Compare the most/least confident images of a specific digit. Filter the test dataset to containing only images where the true digit is 7. Using the `plot_images_grid` function plot the 64 images we are most and least confident about their prediction. Can you identify any differences between these sets?
10. In this question, we show the runtime differences between SGD and GD. Using the same

architecture defined at the beginning of this section, train a network twice: once using GD as the solver and one using SGD as the solver.

- Initialize both solvers to use a fixed learning rate of 10^{-1} , a maximum number of 10,000 iterations and a tolerance of 10^{-10} .
- When using the SGD solver use batches of 64 samples.
- Specify a callback function to record at each iteration the network's current loss and time passed from the beginning of the fit.
- Train the networks over the first 2500 train samples

Then, plot the following:

- For each solver separately, plot a graph showing the running time vs. loss. (i.e. two figures)
- An additional figure with graphs of both solvers one on top of the other as two different scatters (i.e. a single figure, no subplots, two scatters)

Explain the similarities and differences between the fit processes using the two solvers. Consider running times, loss scales and shape of curve.